

머신러닝 데이터셋 추천 가이드

— 분류 · 회귀 · 군집 그리고 다목적 데이터셋 —

Python · scikit-learn 으로 바로 부를 수 있는 데이터셋 중심

1. 분류(Classification) 대표 데이터셋

분류 문제는 머신러닝 입문의 첫 관문이다. 입력 X 와 범주형 라벨 y 가 짝지어진 데이터로, 모델이 새로운 X 를 보고 y 를 맞추도록 학습시킨다. 강의용 데이터셋은 (1) 클래스가 너무 많지 않고, (2) 변수 개수가 시각화 가능한 범위 안에 있으며, (3) 클래스 균형이 적당해야 학습자가 결과를 직관적으로 이해할 수 있다.

데이터셋	규모	특징	호출 방법
Iris	150 × 4 / 3-class	가장 유명. 3 종의 붓꽃 분류, 거의 선형 분리 가능	load_iris()
Breast Cancer Wisconsin	569 × 30 / 2-class	유방암 양성·악성 진단. 의료 데이터 입문	load_breast_cancer()
Digits (MNIST mini)	1,797 × 64 / 10-class	8×8 손글씨 숫자. 다중 클래스 입문에 최적	load_digits()
Wine	178 × 13 / 3-class	와인 화학 성분으로 품종 분류	load_wine()
Titanic	891 × 12 / 2-class	생존 예측. 결측치·범주형 변수 처리 학습용	seaborn.load_dataset('titanic')
MNIST 전체	70,000 × 784 / 10-class	본격 손글씨. CNN·딥러닝의 표준	tensorflow.keras.datasets.mnist
Adult (Census Income)	32,561 × 14 / 2-class	인구통계로 연 수입 50K 초과 여부 예측	fetch_openml('adult')

[그림 1] 분류용 대표 데이터셋 7 종

추천 1 순위 – Iris

클래스 균형(50·50·50), 변수 4 개, 모든 산점도가 한 화면에 들어와 입문 강의의 표준이다. setosa 는 선형 분리 가능, versicolor 와 virginica 는 약간 겹침 – "거의 풀리는데 완전히는 아닌" 교육적으로 가장 좋은 난이도를 준다. 본 강의 시리즈에서도 K-Means · 로지스틱 회귀에 모두 등장.

sklearn 분류 데이터셋 호출 예시 – 모두 동일한 패턴

```
from sklearn.datasets import load_iris, load_breast_cancer, load_digits, load_wine
```

```
iris = load_iris()
print(iris.data.shape, iris.target.shape, iris.target_names)
# (150, 4) (150,) ['setosa' 'versicolor' 'virginica']
```

```
cancer = load_breast_cancer()
```

```
digits = load_digits()
wine = load_wine()
```

2. 회귀(Regression) 대표 데이터셋

회귀는 연속값 y 를 예측하는 문제다. 분류와 달리 정답이 무한히 많은 셈이라, 평가 지표도 MSE·MAE· R^2 같은 거리 기반 지표를 쓴다. 회귀 데이터셋을 고를 때는 변수 간 다중공선성이 적당히 있고, 종속변수의 분포가 너무 한쪽으로 치우치지 않은 것이 좋다.

데이터셋	규모	특징	호출 방법
Diabetes	442 × 10 / 연속	1 년 후 당뇨 진행 정도 예측. Boston 의 공식 대체	load_diabetes()
California Housing	20,640 × 8 / 연속	캘리포니아 주택 가격. Boston 의 후계자	fetch_california_housing()
Auto MPG	398 × 8 / 연속	자동차 연비 예측. 다중회귀·범주형 혼합	seaborn.load_dataset('mpg')
Tips	244 × 7 / 연속	식당 팁 액수. 입문 단순회귀 실습용	seaborn.load_dataset('tips')
Ames Housing	2,930 × 80 / 연속	Boston 의 본격 대체. 특성 공학 강의용	Kaggle 'House Prices'
Energy Efficiency	768 × 8 / 연속 2 개	건물의 난방·냉방 부하. 다중출력 회귀	UCI repository

[그림 2] 회귀용 대표 데이터셋 6 종

주의 – Boston Housing 사용 금지

인종 변수($B = \text{"흑인 비율의 함수"}$)의 윤리적 문제로 sklearn 1.2 부터 제거되었다.

강의 자료에서 절대 쓰지 말고, 다음 두 가지로 대체한다.

- 변수 10 개·샘플 442 개의 가벼운 입문 – Diabetes
- Boston 과 같은 주택 가격 도메인 – California Housing 또는 Ames Housing

sklearn 회귀 데이터셋 호출 예시

```
from sklearn.datasets import load_diabetes, fetch_california_housing
```

1) Diabetes – Boston 대체 1 순위

```
db = load_diabetes()
print(db.data.shape, db.target.shape) # (442, 10) (442,)
```

2) California Housing – 본격 주택 가격 회귀

```
ca = fetch_california_housing()
print(ca.data.shape, ca.target.shape) # (20640, 8) (20640,)
print(ca.feature_names)
# ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
# 'AveOccup', 'Latitude', 'Longitude']
```

3. 군집(Clustering) 대표 데이터셋

군집은 정답 라벨이 없는 비지도학습이 본질이라, 실무에선 보통 자체 데이터를 쓴다. 하지만 강의 검증용으로는 (1) 합성 데이터로 알고리즘 동작을 시각화하고, (2) 라벨이 있는 데이터의 라벨을 잠시 떼서 군집한 뒤 라벨과 비교하는 방식이 표준이다.

데이터셋	규모	특징	호출 방법
make_blobs	임의	가우시안 군집 합성. K-Means 시각화의 표준	make_blobs(n_samples=300, centers=4)
make_moons	임의	K-Means 의 약점·DBSCAN 의 강점을 보이는 고전	make_moons(noise=0.05)
make_circles	임의	비선형 군집. DBSCAN·Spectral 시연용	make_circles(factor=0.5)
Mall Customers	200 × 5	쇼핑몰 고객 세분화. 실무형 군집 입문	Kaggle 'Mall Customer'
Online Retail	541,909 × 8	영국 온라인 소매. RFM + 군집 표준	UCI repository
Wholesale Customers	440 × 8	도매업체 고객 구매 패턴. PCA + K-Means 강의	UCI repository

[그림 3] 군집용 대표 데이터셋 6 종

천 1 순위 – make_blobs + Mall Customers 의 콤보

먼저 make_blobs 의 합성 데이터로 K-Means 가 어떻게 중심을 잡아가는지 단계별 시각화를 보이고, 다음에 Mall Customers 같은 실제 데이터로 활용 사례를 연결한다.

"합성 → 실데이터" 의 흐름이 학습자가 가장 빠르게 감을 잡는 패턴이다.

합성 데이터로 군집 시각화

```
from sklearn.datasets import make_blobs, make_moons, make_circles
from sklearn.cluster import KMeans, DBSCAN
import matplotlib.pyplot as plt
```

1) make_blobs – K-Means 가 잘 푸는 모양

```
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)
labels = KMeans(n_clusters=4, n_init=10, random_state=42).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis'); plt.show()
```

2) make_moons – K-Means 가 못 풀고 DBSCAN 이 잘 푸는 모양

```
X, _ = make_moons(n_samples=300, noise=0.05, random_state=0)
labels = DBSCAN(eps=0.2, min_samples=5).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis'); plt.show()
```

4. 세 작업 모두에 쓸 수 있는 다목적 데이터셋

같은 데이터로 분류·회귀·군집을 모두 해 보면 학생이 "같은 데이터를 다른 관점으로 본다" 는 감을 잡는 데 큰 도움이 된다. 머신러닝의 세 가지 큰 갈래가 어떻게 데이터의 다른 면을 보는지를 한 차시 안에서 보여 줄 수 있어 매우 강력한 교수 도구다.

4.1 Iris (★★★ 추천 1 순위)

Iris 데이터의 강력함은 4 개 변수만으로도 분류·회귀·군집을 모두 자연스럽게 다룰 수 있다는 점에 있다. 변수가 많지 않아 산점도 행렬 한 장으로 데이터 전체를 보여 줄 수 있고, 클래스 균형이 완벽해 어떤 작업을 하든 결과 해석이 깔끔하다.

작업	활용 방식
분류	species (3 종)를 타깃으로 3-class 분류 — 가장 흔한 사용법
회귀	petal length 를 타깃으로 나머지 3 개 변수로 예측 — 단순 다중회귀
군집	라벨을 떼고 4 개 변수로 K-Means(k=3) → 실제 종 라벨과 ARI 로 비교

[그림 4] Iris 한 데이터로 세 작업을 모두 다루는 방식

```
# Iris 로 분류·회귀·군집을 한 번에
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, mean_squared_error, adjusted_rand_score
import numpy as np

iris = load_iris()
X = iris.data; y = iris.target
X_s = StandardScaler().fit_transform(X)

# 1) 분류 - petal/sepal 로 species 맞추기
clf = LogisticRegression(max_iter=200).fit(X_s, y)
print('분류 정확도 :', accuracy_score(y, clf.predict(X_s)).round(3))

# 2) 회귀 - petal length 를 다른 3 변수로 예측
target_idx = 2 # petal length
X_r = np.delete(X_s, target_idx, axis=1)
y_r = X[:, target_idx]
reg = LinearRegression().fit(X_r, y_r)
print('회귀 MSE :', mean_squared_error(y_r, reg.predict(X_r)).round(3))

# 3) 군집 - 라벨 떼고 K-Means(k=3) → 실제 라벨과 ARI 비교
km = KMeans(n_clusters=3, n_init=20, random_state=42).fit(X_s)
print('군집 ARI :', adjusted_rand_score(y, km.labels_).round(3))
```

4.2 Wine

Wine 은 Iris 와 비슷한 구조이면서 변수가 13 개로 좀 더 풍부하다. 화학 성분 변수들이 들어 있어 "도메인 지식이 모델에 어떻게 녹는가" 를 함께 다루기 좋다. 변수 간 상관관계도 적당히 있어 다중공선성·정규화 강의의 좋은 재료가 된다.

작업	활용 방식
분류	cultivar (3 종)를 타깃으로 분류
회귀	alcohol 함량을 나머지 12 개 변수로 예측
군집	라벨 떼고 화학 성분 13 개로 K-Means(k=3) → ARI 비교

[그림 5] Wine 다목적 활용

4.3 Diabetes

Diabetes 는 본래 회귀 데이터지만, 진행도를 중앙값으로 잘라 이진 라벨을 만들면 분류로도 쓸 수 있다. 의료 도메인 데이터라는 점에서 군집화도 자연스럽게 의미가 있다 — "비슷한 진행 패턴을 가진 환자군" 을 찾는 작업이다.

작업	활용 방식
회귀	1 년 후 당뇨 진행도(연속값) 예측 — 본래 용도
분류	진행도의 중앙값으로 자른 이진 라벨로 분류
군집	환자들을 특성으로 묶기 — 의료 도메인 군집 사례

[그림 6] Diabetes 다목적 활용

4.4 California Housing

본격 회귀 데이터지만 다양하게 변형할 수 있다. 집값을 사분위로 잘라 4-class 분류로 바꾸거나, 위·경도로 지역 군집화를 해 보면 캘리포니아 주택 시장의 지역적 구조가 드러난다.

작업	활용 방식
회귀	집값(MedHouseVal) 예측 — 본래 용도
분류	집값 사분위로 자른 4-class 또는 중앙값으로 이진 분류
군집	위·경도 + 인구 통계로 지역 군집화 — 실무형 사례

[그림 7] California Housing 다목적 활용

4.5 Mall Customers

본래는 군집 데이터지만, 군집화 결과를 라벨로 삼으면 지도학습 문제로도 바꿀 수 있다. "비지도학습 → 라벨 생성 → 지도학습" 의 흐름은 실무에서 매우 자주 쓰는 패턴이라 학생에게 꼭 보여 줄 가치가 있다.

작업	활용 방식
군집	본래 용도 — 5 개 변수로 고객 세분화 (Spending Score, Income, Age 등)
분류	군집 결과를 라벨로 삼아 새 고객의 군집 예측 (지도학습 변환)
회귀	Spending Score 를 나머지 변수로 예측

[그림 8] Mall Customers 다목적 활용

5. 강의 자료 작성 관점의 권장 조합

데이터셋을 한 곳에 모으는 강의를 설계할 의향이 있다면 다음 세 흐름을 추천한다. 각 흐름은 학습자의 단계와 강의 목적에 따라 골라 쓰면 된다.

5.1 분류·회귀·군집 비교 차시 (입문)

Iris 한 데이터로 세 작업을 모두 해 보고, sklearn API 가 어떻게 같은 패턴(fit → predict / predict_proba / labels_)을 공유하는지 보여 준다. 한 시간 분량으로 머신러닝의 큰 그림을 심을 수 있는 매우 효율적인 구성이다.

5.2 실무형 보충 차시 (중급)

Mall Customers 또는 Online Retail 로 비즈니스 군집·세분화를 한 다음, 군집 라벨을 타킷으로 분류 모델까지 확장한다. "비지도학습으로 라벨을 만들고 지도학습으로 확장" 하는 실무 패턴을 몸으로 익히게 한다.

5.3 회귀 심화 차시 (심화)

Boston 의 빈자리를 California Housing 으로 채우고, Ames Housing 의 80 개 변수로 본격 EDA·특성 공학을 다룬다. 이상치 처리·로그 변환·범주형 인코딩 같은 실전 기법을 두루 다루기 좋다.

6. sklearn 데이터셋 API 정리

sklearn 의 데이터셋 함수는 크게 세 종류로 나뉜다. 이 구분을 알고 있으면 새 데이터셋을 만났을 때도 어떻게 부르는지 빠르게 추정할 수 있다.

sklearn 데이터셋 함수 세 종류

- load_* - 패키지에 포함된 작은 데이터셋. 다운로드 없이 즉시 사용. (Iris, Wine, Diabetes 등)
- fetch_* - 큰 데이터셋. 첫 호출 시 다운로드, 캐시 후 재사용. (California Housing, openml 등)
- make_* - 합성 데이터 생성기. 파라미터로 모양·크기를 조절. (make_blobs, make_moons 등)

세 종류를 한 화면에서

```
from sklearn.datasets import load_iris # 패키지 내장
from sklearn.datasets import fetch_california_housing # 자동 다운로드
from sklearn.datasets import make_blobs # 합성 생성
```

```
iris = load_iris()
ca = fetch_california_housing()
X, y = make_blobs(n_samples=300, centers=4, random_state=42)
```

```
# 공통 패턴 - Bunch 객체로 .data / .target / .feature_names / .target_names
print(iris.feature_names)
# ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

7. 정리 및 요약

- 분류 입문 — Iris (3 종 50 개씩 균형, 변수 4 개). 의료 도메인은 Breast Cancer, 이미지 입문은 Digits.
- 회귀 입문 — Diabetes (Boston 의 공식 대체). 본격 주택 가격은 California Housing.
- Boston Housing 금지 — 인종 변수의 윤리 문제로 sklearn 1.2 에서 제거됨. Diabetes·California 로 대체.
- 군집 시각화 — make_blobs (성공 사례) 와 make_moons (K-Means 실패 사례) 콤보가 강의용 표준.
- 실무형 군집 — Mall Customers (고객 세분화) 또는 Online Retail (RFM 분석).

- 다목적 1 순위 — Iris. 분류·회귀(petal length 예측)·군집(K-Means k=3) 을 한 차시에 모두 다룰 수 있다.
- 비지도 → 지도 변환 — Mall Customers 처럼 군집 결과를 라벨로 삼아 분류 모델로 확장하는 패턴.
- sklearn API 세 갈래 — load_* (내장) / fetch_* (다운로드) / make_* (합성). 호출 패턴은 동일.

과제 해답

1 번 해답

```
from sklearn.datasets import load_iris, load_diabetes, make_blobs

# 분류
iris = load_iris()
print('Iris      :', iris.data.shape, iris.target.shape)
# Iris      : (150, 4) (150,)

# 회귀
db = load_diabetes()
print('Diabetes  :', db.data.shape, db.target.shape)
# Diabetes   : (442, 10) (442,)

# 군집 (합성)
X, _ = make_blobs(n_samples=300, centers=4, random_state=42)
print('make_blobs:', X.shape)
# make_blobs: (300, 2)
```

선택 이유 한 줄씩 — Iris: 변수 4 개·클래스 균형으로 입문 표준. Diabetes: 변수 10 개·샘플 442 개로 Boston 의 가벼운 대체. make_blobs: 군집 모양과 개수를 자유롭게 조절해 알고리즘 시각화에 최적.

2 번 해답

Boston Housing 은 인종 변수(B = 흑인 비율의 함수) 가 들어 있어 윤리적으로 부적절하다는 비판이 누적되어 sklearn 1.2 부터 제거되었다. 대체 데이터셋은 다음 두 가지가 표준이다.

- Diabetes — load_diabetes() — 가벼운 입문용 (442×10)
- California Housing — fetch_california_housing() — 본격 주택 가격 (20640×8)

3 번 해답

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, mean_squared_error, adjusted_rand_score
import numpy as np

iris = load_iris(); X = iris.data; y = iris.target
X_s = StandardScaler().fit_transform(X)

# (1) 분류
clf = LogisticRegression(max_iter=200).fit(X_s, y)
acc = accuracy_score(y, clf.predict(X_s))

# (2) 회귀 - petal length(인덱스 2) 예측
X_r = np.delete(X_s, 2, axis=1); y_r = X[:, 2]
reg = LinearRegression().fit(X_r, y_r)
mse = mean_squared_error(y_r, reg.predict(X_r))

# (3) 군집
km = KMeans(n_clusters=3, n_init=20, random_state=42).fit(X_s)
ari = adjusted_rand_score(y, km.labels_)
```



```
print(f'분류 정확도={acc:.3f}, 회귀 MSE={mse:.3f}, 군집 ARI={ari:.3f}')
```

비교 해석: 분류 정확도는 0.97 안팎으로 매우 높고(species 는 변수와 거의 선형 관계), 회귀 MSE 는 petal length 가 다른 변수들과 강한 상관을 가져 작게 나오며, 군집 ARI 는 0.6~0.7 수준으로 versicolor·virginica 가 겹치는 부분에서 일부 오분류가 생긴다. 같은 데이터를 보는 세 시각이 모두 의미 있는 정보를 준다.

4 번 해답

```
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, DBSCAN
import matplotlib.pyplot as plt

X, _ = make_moons(n_samples=300, noise=0.05, random_state=0)

km_lab = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(X)
db_lab = DBSCAN(eps=0.2, min_samples=5).fit_predict(X)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X[:, 0], X[:, 1], c=km_lab, cmap='viridis')
axes[0].set_title('KMeans on Moons')
axes[1].scatter(X[:, 0], X[:, 1], c=db_lab, cmap='viridis')
axes[1].set_title('DBSCAN on Moons')
plt.tight_layout(); plt.show()
```

DBSCAN 이 두 초승달을 자연스럽게 분리한다. K-Means 는 거리 기반으로 중심에서 가까운 점들을 모으는 구형 가정을 깔고 있어 초승달의 두 끝이 서로 멀어 보이므로 한 군집으로 묶기 어렵다. 반면 DBSCAN 은 밀도가 이어진 점들을 따라 군집을 확장하므로 초승달 모양 그대로를 한 군집으로 잡는다. 데이터의 모양이 알고리즘 선택을 결정한다는 좋은 예다.

5 번 해답

```
# Mall Customers - Kaggle 에서 받은 csv 라고 가정
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv('Mall_Customers.csv')
X = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].values
X_s = StandardScaler().fit_transform(X)

# 1) 비지도 - K-Means(k=5) 로 라벨 생성
km = KMeans(n_clusters=5, n_init=20, random_state=42).fit(X_s)
y = km.labels_

# 2) 지도 - 그 라벨로 분류 모델 학습
X_tr, X_te, y_tr, y_te = train_test_split(X_s, y, test_size=0.3, random_state=0)
clf = LogisticRegression(max_iter=500, multi_class='multinomial').fit(X_tr, y_tr)
print('새 고객 분류 정확도:', accuracy_score(y_te, clf.predict(X_te)).round(3))
```

"비지도 → 지도" 변환이 실무에서 유용한 두 가지 이유: (1) 새 고객 데이터가 들어왔을 때 다시 K-Means 를 돌리지 않고도 학습된 분류기로 즉시 군집을 예측할 수 있다 — 서비스 적용이 빠르다. (2) 분류기의 계수·특성 중요도를 통해 "어떤 변수가 군집을 가르는가" 를 해석할 수 있다 — 군집 결과의 비즈니스적 의미가 명확해진다.